

알고리즘 교재

그리디(Greedy) 정리

원문 PDF의 북마크 기준으로 확인된 그리디 장 구성: 15 Greedy Algorithms, 15.1 Activity-selection problem, 15.2 Elements of the greedy strategy, 15.3 Huffman codes, 15.4 Offline caching

한 줄 핵심: 그리디는 매 순간 가장 좋아 보이는 선택을 바로 하는 방법이다. 다만 아무 문제에서나 맞는 것은 아니고, **지금의 최선 선택이 전체 최적해로 이어지는 구조가 있을 때만 통한다.**

1. 그리디란 무엇인가

그리디 알고리즘은 문제를 여러 단계로 나누고, 각 단계에서 **그 순간 가장 이득이 큰 선택**을 하는 방식이다. 한 번 선택한 것은 보통 되돌리지 않는다. 그래서 구현이 단순하고 빠른 경우가 많다.

그리디의 장점

- 생각과 구현이 비교적 단순하다.
- 대개 정렬 후 선형 탐색 형태라 속도가 빠르다.
- 맞는 문제에서는 동적 계획법보다 훨씬 간단하다.

그리디의 위험

- 눈앞의 최선이 전체 최선이 아닐 수 있다.
- 문제 구조를 증명하지 않으면 오답이 되기 쉽다.

즉, 그리디는 **빠른 직감**이 아니라 **증명된 직감**이어야 한다.

2. 그리디가 통하려면

2.1 최적 부분 구조

문제의 최적해가 더 작은 부분 문제들의 최적해로부터 만들어질 수 있어야 한다. 이는 동적 계획법에서도 자주 보이는 조건이다.

2.2 그리디 선택 속성

어떤 단계에서 **지금 당장 최선으로 보이는 선택 하나**를 해도, 여전히 남은 문제에서 최적해를 만들 수 있어야 한다. 즉, “맨 처음 선택을 욕심내게 해도 괜찮다”는 것이 핵심이다.

동적 계획법과 비교

- 동적 계획법: 가능한 선택을 넓게 비교해서 최적해를 조합한다.
- 그리디: 한 단계에서 좋은 선택을 바로 확정한다.

그래서 그리디는 보통 더 빠르지만, 적용 가능 범위는 더 좁다.

3. 15.1 활동 선택 문제

여러 활동이 있고, 각 활동은 시작 시간과 끝나는 시간이 있다. 서로 겹치지 않게 최대한 많은 활동을 선택하는 문제가 활동 선택 문제다.

왜 “가장 빨리 끝나는 활동”을 먼저 고를까

끝나는 시간이 가장 빠른 활동을 먼저 고르면, 그 뒤에 들어갈 수 있는 시간이 가장 많이 남는다. 즉, 이후 선택의 자유가 최대가 된다.

핵심 아이디어

- 활동을 종료 시간 기준으로 정렬한다.
- 가장 먼저 끝나는 활동을 선택한다.
- 그 활동과 겹치지 않는 다음 활동 중 가장 먼저 끝나는 것을 계속 선택한다.

이 문제는 그리디 선택 속성이 성립하는 대표 예시다. 처음에 가장 빨리 끝나는 활동을 골라도 최적해를 잃지 않는다는 것을 교환 논법으로 증명할 수 있다.

시간복잡도: 보통 정렬 때문에 $O(n \log n)$, 정렬 후 선택은 $O(n)$ 이다.

4. 15.2 그리디 전략의 요소

이 절은 “그리디를 어떻게 설계하고, 왜 맞는지 어떻게 증명하는가”를 설명하는 중심 파트다.

설계 흐름

- 문제를 단계별 선택 문제로 본다.
- 어떤 선택이 가장 유망한지 규칙을 세운다.
- 그 선택을 해도 최적해를 잃지 않는다는 것을 증명한다.
- 선택 후 남는 문제가 원래 문제와 같은 형태인지 확인한다.

증명에서 자주 쓰는 방식

- **교환 논법**: 최적해가 내 선택과 다르더라도, 내 선택으로 바뀌도 손해가 없음을 보인다.
- **남은 문제 축소**: 첫 선택 후의 문제도 같은 구조의 최적화 문제임을 보인다.

재혁님용 감각

그리디는 "그냥 이게 좋아 보여서"가 아니다. 항상 **왜 지금 이 선택을 먼저 해도 되는지**를 설명할 수 있어야 한다.

5. 15.3 허프만 코드

허프만 코드는 문자마다 등장 빈도가 다를 때, 자주 나오는 문자는 짧은 비트열로, 덜 나오는 문자는 긴 비트열로 표현해서 전체 저장 공간을 줄이는 방법이다.

핵심 개념

- **접두어 코드(prefix code)**: 어떤 코드도 다른 코드의 앞부분이 되지 않아서 모호성이 없다.
- **빈도 기반 압축**: 자주 나오는 문자를 짧게 만들면 전체 평균 길이가 줄어든다.

그리디 선택은 무엇인가

빈도가 가장 작은 두 문자를 먼저 묶는다. 이 둘은 가장 깊은 곳으로 보내는 것이 유리하기 때문이다. 이 과정을 반복해서 트리를 만든다.

알고리즘 흐름

- 각 문자를 빈도와 함께 우선순위 큐에 넣는다.
- 가장 빈도가 작은 두 노드를 꺼낸다.
- 둘을 자식으로 갖는 새 노드를 만들고, 빈도 합으로 다시 넣는다.
- 노드가 하나 남을 때까지 반복한다.

완성된 이진 트리에서 왼쪽을 0, 오른쪽을 1로 두면 각 문자의 비트열이 만들어진다.

시간복잡도: 우선순위 큐를 쓰면 보통 $O(n \log n)$ 이다.

6. 15.4 오프라인 캐싱

캐시는 자주 쓰는 데이터를 빠른 저장공간에 보관하는 장치다. 오프라인 캐싱은 **앞으로 어떤 요청들이 들어올지 전부 알고 있다고 가정하는 이상적인 모델**이다.

문제 목표

캐시 크기가 제한된 상황에서, 캐시 미스를 최소화하도록 어떤 데이터를 내보낼지 결정한다.

그리디 선택은 무엇인가

캐시가 꼭 찼을 때는 **앞으로 가장 늦게 다시 사용될 항목**을 내보낸다. 혹은 다시는 안 쓰일 항목이 있으면 그것을 내보내면 된다.

왜 이게 맞나

곧 다시 써야 하는 항목을 버리면 가까운 미래에 바로 다시 불러와야 해서 손해다. 반대로 가장 나중에 쓸 항목을 버리면 당장의 미래 구간에는 영향을 덜 준다.

이 전략은 미래를 알고 있을 때의 최적 정책으로 유명하다. 실제 운영체제에서는 미래를 모르기 때문에 근사한 전략을 쓴다.

7. 이 장 전체의 핵심 메시지

- 그리디는 “매 순간 최선 선택”이지만, 아무 데나 쓰면 안 된다.
- 반드시 **그리디 선택 속성과 최적 부분 구조**를 확인해야 한다.
- 활동 선택 문제는 그리디가 잘 맞는 대표 예시다.
- 허프만 코드는 그리디가 실제로 매우 강력하게 쓰이는 대표 사례다.
- 오프라인 캐싱은 “미래를 알면 어떤 탐욕적 선택이 최적인가”를 보여주는 예다.

8. 시험이나 문제풀이에서 어떻게 판단할까

그리디 의심 신호

- 매 단계에서 하나를 확정해도 될 것처럼 보인다.

- 정렬 후 앞에서부터 훑는 구조가 자연스럽다.
- 가장 빠른 것, 가장 작은 것, 가장 큰 것, 가장 싼 것 같은 기준이 보인다.

하지만 꼭 확인할 것

- 첫 선택을 바꿔도 손해가 없다는 증명이 되는가
- 첫 선택 후 남는 문제가 같은 형태로 줄어드는가

9. 한 페이지 압축 암기

- 그리디 = 지금 가장 좋아 보이는 선택을 바로 확정하는 방법
- 조건 = 최적 부분 구조 + 그리디 선택 속성
- 활동 선택 = 가장 빨리 끝나는 활동부터 고른다
- 허프만 코드 = 가장 빈도 작은 두 개를 계속 묶는다
- 오프라인 캐싱 = 가장 늦게 다시 쓰일 데이터를 버린다
- 증명 포인트 = 교환 논법으로 "내 선택을 먼저 해도 괜찮다" 보이기

이 문서는 재혁님이 올려주신 알고리즘 교재 PDF의 그리디 장 북마크 구조를 기준으로, 초보자 관점에서 핵심 개념과 예시를 다시 풀어 쓴 요약본입니다.